Water Pipe Leakage Detection System

Microproject Report

Submitted by:

Adhil M M

S4 EV | MGP24EVLSI005

Project Repository: Water Pipe Leakage Detection

1. Introduction and Objectives

Water pipe leakage is a major cause of water wastage in both residential and industrial settings. Early detection of leaks can prevent significant loss of resources and reduce damage to infrastructure. This project aims to detect water pipe leakage in real time using an ESP32 microcontroller and two YFS201 water flow sensors placed at the inlet and outlet of a pipe. When a discrepancy between the two flow readings is detected, the system triggers a local alert and sends a notification via the Blynk IoT platform.

Objectives:

- Measure and monitor water flow rate in real time at both inlet and outlet.
- Detect leakage by comparing inlet and outlet flow differentials.
- Display live readings on a 16x2 LCD display.
- Trigger a buzzer alert immediately upon leak detection.
- Send real-time data and push notifications to the Blynk IoT app.
- Support water conservation and pipe infrastructure monitoring.

2. Block Diagram and Explanation

Two YF-S201 water flow sensors are installed at the inlet and outlet of the monitored pipe segment. Both sensors send pulse signals to the ESP32, which calculates the flow rate in litres per minute. If the inlet flow significantly exceeds the outlet flow (indicating a leak), the ESP32 activates the buzzer and displays a warning on the 16x2 LCD. Simultaneously, it sends the flow data and a leak alert to the Blynk cloud via WiFi, which pushes a notification to the user's smartphone.

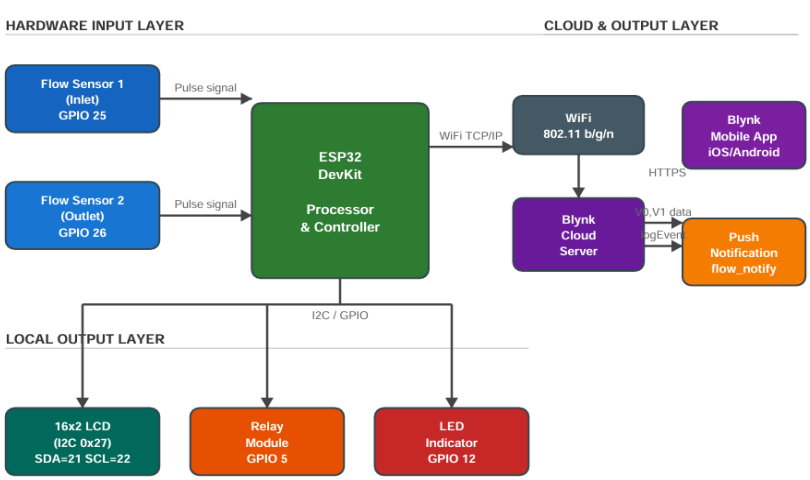

Figure 1: Block Diagram of Water Pipe Leakage Detection System

3. Circuit Diagram and Components

Components Used:

- ESP32 DevKit (ESP32-WROOM-32)
- YF-S201 Water Flow Sensor × 2 (Inlet and Outlet)
- 16x2 LCD Display with I2C Module (PCF8574)
- Active Buzzer / Relay Module (5V)
- LED Indicator
- Jumper Wires and Breadboard
- 5V / 2A USB Power Supply

Interfacing / Wiring Connections:

Component	Pin / Wire	ESP32 Pin
Flow Sensor 1 (Inlet)	VCC (Red)	VIN (5V)
Flow Sensor 1 (Inlet)	GND (Black)	GND
Flow Sensor 1 (Inlet)	Signal (Yellow)	GPIO 25
Flow Sensor 2 (Outlet)	VCC (Red)	VIN (5V)
Flow Sensor 2 (Outlet)	GND (Black)	GND
Flow Sensor 2 (Outlet)	Signal (Yellow)	GPIO 26
16x2 LCD (I2C)	VCC	VIN (5V)
16x2 LCD (I2C)	GND	GND
16x2 LCD (I2C)	SDA	GPIO 21
16x2 LCD (I2C)	SCL	GPIO 22
Buzzer / Relay	Positive (+)	GPIO 5
LED	Positive (+)	GPIO 12
LED	Negative (–)	GND

Figure 2: Wiring / Interfacing Table

Working Principle: The YF-S201 flow sensor uses a hall-effect sensor and a spinning turbine to generate pulses proportional to water flow. The ESP32 counts these pulses using hardware interrupts and converts them into flow rate (mL/s) using the formula: Flow Rate = Pulse Count / (Calibration Factor × time interval). If the outlet flow rate falls below the inlet flow rate and outlet is below 8 mL/s, a leak is flagged. The 16x2 LCD is driven via I2C using the LiquidCrystal_I2C library, displaying both sensor readings and a LEAK DETECTED message when triggered.

4. Code and Interfacing Logic

Libraries Used (Arduino IDE):

- Blynk — by Volodymyr Shymanskyy (cloud communication and push notifications)
- LiquidCrystal_I2C — by Frank de Brabander (16x2 LCD via I2C)
- WiFi — built-in with ESP32 Arduino core (wireless connectivity)

Code Description:

1. Initializes WiFi connection and authenticates with the Blynk cloud using the Auth Token.
2. Attaches hardware interrupts on GPIO 25 and GPIO 26 to count pulses from both flow sensors.
3. Every 1 second, calculates flow rate (mL/s) for inlet and outlet sensors from pulse counts.

4. Displays inlet and outlet flow rates on the 16x2 LCD in real time.

5. Sends both flow rate values to Blynk via Virtual Pins V0 (inlet) and V1 (outlet).

6. Computes the flow difference. If outlet flow drops below inlet and outlet < 8 mL/s, leak is detected.

7. On leak detection: activates relay (GPIO 5) and LED (GPIO 12), displays LEAKAGE DETECTED on LCD.

8. Sends a Blynk logEvent 'flow_notify' notification to the user's smartphone.

9. Supports manual motor control via Blynk V2 (ON/OFF button) and V3 (Auto/Manual mode toggle).

10. When flow returns to normal in AUTO mode, resets relay and LED (all clear).

Blynk Virtual Pin Mapping:

Virtual Pin	Name	Data Type	Purpose
V0	Inlet Flow Rate	Double	Inlet sensor flow rate (mL/s)
V1	Outlet Flow Rate	Double	Outlet sensor flow rate (mL/s)
V2	Motor Control	Integer	1 = Motor ON, 0 = Motor OFF (manual mode)
V3	Mode Toggle	Integer	1 = Manual Override, 0 = Auto Mode

Full Arduino Source Code:

```
#define BLYNK_TEMPLATE_ID    "TMPL3clVIW9Hp"
#define BLYNK_TEMPLATE_NAME  "water pipe leakage monitoring system"
#define BLYNK_AUTH_TOKEN     "3dT7_sDlclNretj251qbm4CPAUIa7rSg"
#define BLYNK_PRINT          Serial

#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---- Pin Definitions ----
#define RELAY    5
#define SENSOR1  25
#define SENSOR2  26
#define LED_PIN  12

// ---- LCD Setup ----
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ---- WiFi & Blynk ----
char auth[] = BLYNK_AUTH_TOKEN;
char ssid[] = "Prrr";
char pass[] = "87654321";

// ---- Sensor Variables ----
long previousMillis1 = 0;
long previousMillis2 = 0;
int interval = 1000;

float calibrationFactor1 = 6;
float calibrationFactor2 = 6;

volatile byte pulseCount1 = 0;
volatile byte pulseCount2 = 0;
```

```
float flowRate1 = 0.0, flowRate2 = 0.0;

// ---- Manual Override Variables ----
bool manualOverride = false;
bool manualRelayState = false;

// ---- Interrupts ----
void IRAM_ATTR pulseCounter1() { pulseCount1++; }
void IRAM_ATTR pulseCounter2() { pulseCount2++; }

// V2 — Motor ON/OFF button (only works when manual mode is ON)
BLYNK_WRITE(V2)
{
  manualRelayState = param.asInt();
  if (manualOverride)
  {
    digitalWrite(RELAY, manualRelayState ? HIGH : LOW);
    digitalWrite(LED_PIN, manualRelayState ? HIGH : LOW);
    lcd.setCursor(0, 0);
    lcd.print(manualRelayState ? "Motor: ON (Man) " : "Motor: OFF(Man) ");
  }
}

// V3 — Manual/Auto mode toggle button
BLYNK_WRITE(V3)
{
  manualOverride = param.asInt();
  lcd.setCursor(0, 1);
  lcd.print(manualOverride ? "Mode: MANUAL    " : "Mode: AUTO      ");
  if (!manualOverride)
  {
    digitalWrite(RELAY, LOW);
    digitalWrite(LED_PIN, LOW);
  }
  Serial.print("Manual Override: ");
  Serial.println(manualOverride ? "ON" : "OFF");
}

void setup()
{
  Serial.begin(115200);
  pinMode(SENSOR1, INPUT_PULLUP);
  pinMode(SENSOR2, INPUT_PULLUP);
  pinMode(RELAY, OUTPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(RELAY, LOW);
  digitalWrite(LED_PIN, LOW);
  attachInterrupt(digitalPinToInterrupt(SENSOR1), pulseCounter1, FALLING);
  attachInterrupt(digitalPinToInterrupt(SENSOR2), pulseCounter2, FALLING);
  Wire.begin(21, 22);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print(" IoT Water Leak ");
  lcd.setCursor(0, 1);
  lcd.print("   Monitoring   ");
  delay(2000);
  lcd.clear();
  Blynk.begin(auth, ssid, pass);
}

void loop()
{
  Blynk.run();
  unsigned long currentMillis = millis();

  // ---- Sensor 1 ----
```

```
  if (currentMillis - previousMillis1 >= interval)
  {
    noInterrupts();
    byte pulses = pulseCount1;
    pulseCount1 = 0;
    interrupts();
    flowRate1 = (pulses / calibrationFactor1);
    previousMillis1 = currentMillis;
    Serial.print("F1: ");
    Serial.println(flowRate1);
    Blynk.virtualWrite(V0, flowRate1);
    if (!manualOverride)
    {
      lcd.setCursor(0, 0);
      lcd.print("F1:");
      lcd.print((int)flowRate1);
      lcd.print(" mL/s   ");
    }
  }

  // ---- Sensor 2 ----
  if (currentMillis - previousMillis2 >= interval)
  {
    noInterrupts();
    byte pulses = pulseCount2;
    pulseCount2 = 0;
    interrupts();
    flowRate2 = (pulses / calibrationFactor2);
    previousMillis2 = currentMillis;
    Serial.print("F2: ");
    Serial.println(flowRate2);
    Blynk.virtualWrite(V1, flowRate2);
    if (!manualOverride)
    {
      lcd.setCursor(0, 1);
      lcd.print("F2:");
      lcd.print((int)flowRate2);
      lcd.print(" mL/s   ");
    }
  }

  // ---- Leakage Detection (AUTO mode only) ----
  if (!manualOverride)
  {
    if (flowRate2 < flowRate1 && flowRate2 < 8)
    {
      lcd.setCursor(0, 0);
      lcd.print("Leakage Detected");
      lcd.setCursor(0, 1);
      lcd.print("F1 > F2         ");
      digitalWrite(RELAY, HIGH);
      digitalWrite(LED_PIN, HIGH);
      Blynk.logEvent("flow_notify", "Water Leakage Detected");
    }
    else
    {
      digitalWrite(RELAY, LOW);
      digitalWrite(LED_PIN, LOW);
    }
  }
}
```

5. Working Model

The hardware prototype consists of the ESP32 DevKit mounted on a breadboard, connected to two YF-S201 water flow sensors through jumper wires. The 16x2 LCD (I2C) is wired to the ESP32's I2C pins (GPIO 21 for SDA, GPIO 22 for SCL) and displays live flow readings for both inlet and outlet. A relay module connected to GPIO 5 and an LED indicator on GPIO 12 provide alert outputs. The assembled circuit is powered via a 5V USB supply. When the system is running, the Blynk app on the smartphone displays real-time gauge readings for both sensors. A push notification is also sent to the phone immediately upon leak detection. The system also supports manual motor override via the Blynk app.

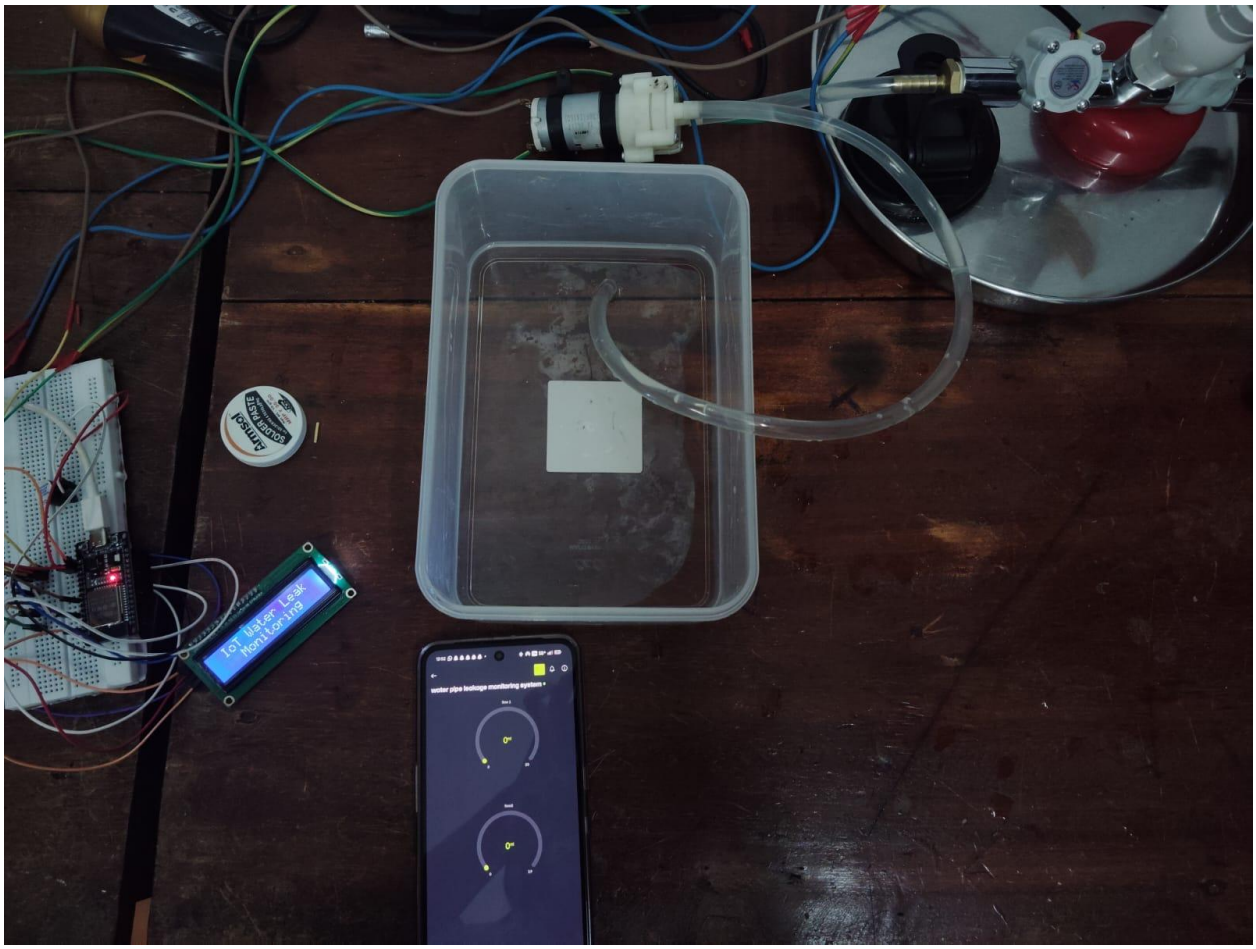

Figure 3: Working Model of the Water Pipe Leakage Detection System

6. Conclusion

The Water Pipe Leakage Detection System successfully demonstrates real-time monitoring and automatic leak detection using IoT technology. The combination of the ESP32 microcontroller, YF-S201 flow sensors, and the Blynk IoT platform offers a cost-effective and scalable solution for water infrastructure monitoring. The system accurately identifies leaks by comparing inlet and outlet flow rates, provides immediate local alerts via the relay and LED indicators, and delivers remote push notifications through the Blynk app. The addition of a manual override feature via the Blynk app further enhances the system's usability and control. This project highlights the potential of IoT in water conservation, smart infrastructure, and preventive maintenance, making it suitable for residential, agricultural, and industrial applications.